

Multi-Agent AI Support Ticket System

Group Project 1 — Agent-to-Agent Communication, RAG, and Browser Automation

Group 2 · Parth Auti, Hasti Bhalodi, Medha Jani, Kento Hopkins · September 28, 2026

1. System Overview

Our system takes a plain-language description of an IT problem and files a completed support ticket for it, without a human choosing the category or writing the resolution notes. It is built from two cooperating agents that communicate over an A2A-style HTTP protocol.

The **Requester Agent** coordinates the workflow. It receives the user's request, submits it to the Specialist as an asynchronous task, polls until the task reaches a terminal state, decides what belongs in each field of the support form, drives a browser to submit it, and then verifies that the confirmation screen reflects what the Specialist actually returned.

The **Specialist Agent** answers the question. It runs a retrieval-augmented generation pipeline over the seven provided support documents: it retrieves candidate passages using two independent retrieval strategies, fuses their rankings, has an LLM grade the survivors for relevance, and generates a resolution grounded only in the retrieved text. It returns a ticket category, the resolution notes, the source files used, and a confidence score.

The Specialist's answer is not merely displayed. The category it returns determines which option is selected in the form's dropdown, and the resolution it generates is the text entered in the resolution field. If the Specialist returns a category the form cannot represent, the Requester refuses to submit rather than filing the ticket incorrectly.

2. Architecture and Workflow

Workflow, step by step

1. A user request arrives through the CLI and enters `handle_request()`.
2. The Requester decides what it needs — a category and a resolution — and submits the issue text as a task via `POST /tasks`.
3. The Specialist registers the task, starts a worker thread, and returns `201` with a task ID and status `submitted` immediately.
4. The Requester polls `GET /tasks/{id}` until the task is terminal.
5. The worker transitions the task to `working` and calls the RAG pipeline.
6. The pipeline retrieves from the knowledge base and generates an answer.
7. The task becomes `completed`; the next poll returns the result.
8. The Requester maps the category to a dropdown value, builds a form plan, and Playwright fills and submits the form.
9. The Requester reads the confirmation and verifies it matches the Specialist's answer.

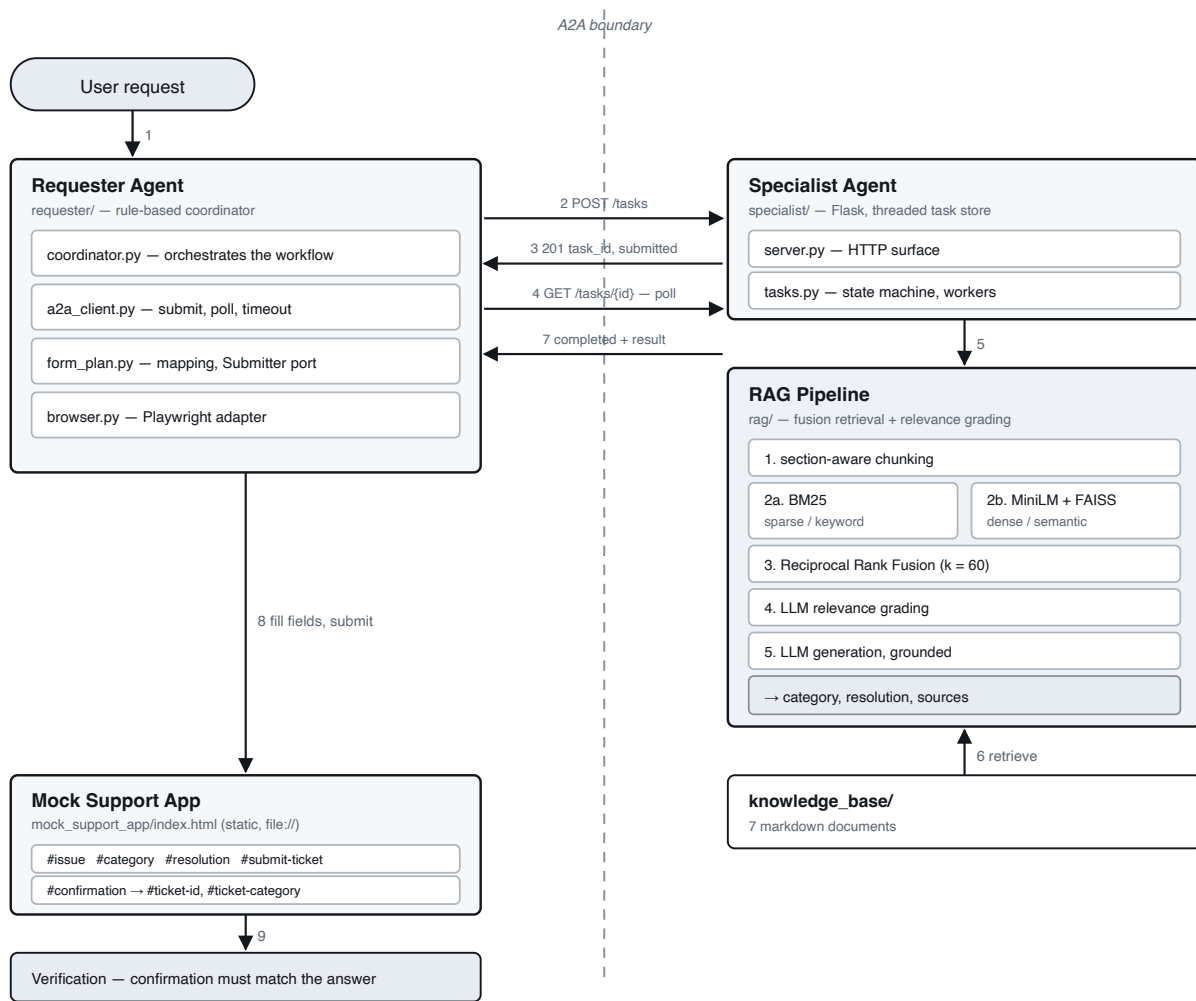


Figure 1 — System architecture and request flow. The dashed line is a real process boundary: the two agents run as separate processes and communicate only over HTTP

Why the components are separated this way

Three boundaries are enforced deliberately, and each one paid for itself during development:

- **The Specialist does not know a web form exists.** It returns the knowledge base's own category wording; translating that into a dropdown value is the Requester's job.
- **The task store does not know RAG exists.** The pipeline is injected as a worker function, so the entire retrieval layer was replaced mid-project without touching the server or the state machine.
- **The coordinator does not know Playwright exists.** It depends on a `Submitter` interface, which let the browser automation be built in parallel with everything else, against a dry-run implementation.

3. The A2A Protocol

We wrote the protocol down in a contracts document before writing any code, and treated changes to it as an event the whole team had to agree on. That decision is the reason four people could build four components in parallel without breaking each other at the seams.

Endpoints

Endpoint	Behaviour
POST /tasks	Submit a task. Responds <code>201</code> immediately with a task ID and status <code>submitted</code> . Does not wait for the work.
GET /tasks/{id}	Current status; carries <code>result</code> once completed, or <code>error</code> if failed. <code>404</code> for an unknown ID.
GET /health	Liveness check.

Task states

State	Terminal	Meaning
<code>submitted</code>	No	Accepted, not yet started.
<code>working</code>	No	The RAG pipeline is running.
<code>completed</code>	Yes	<code>result</code> is present.
<code>failed</code>	Yes	<code>error</code> is present; <code>result</code> is absent.

Messages

```
Requester → Specialist
POST /tasks
{"question": "I forgot my password and cannot log into my account.",
 "needs": ["category", "resolution"]}

Specialist → Requester
201 {"task_id": "a0b059a4-...", "status": "submitted"}

Requester → Specialist      GET /tasks/a0b059a4-...
Specialist → Requester      200 {"status": "working"}

Requester → Specialist      GET /tasks/a0b059a4-...
Specialist → Requester      200 {"status": "completed",
                                "result": {"category": "Account Access",
                                             "resolution": "...",
                                             "sources": ["password_reset.md"],
                                             "confidence": 0.82}}
```

Error taxonomy

Failures are typed rather than returned as free text, so the Requester can react differently to each.

`NO_RELEVANT_CONTEXT` means the knowledge base had nothing useful and no ticket should be filed.

`LLM_ERROR` means the model call failed and a retry might succeed. `UNKNOWN_TASK` and `BAD_REQUEST` are protocol-level faults. `TIMEOUT` and `SPECIALIST_UNREACHABLE` are raised by the client rather than the server.

Design decisions

The acknowledgment is asynchronous by construction. The work runs on a background thread, so `POST /tasks` returns while retrieval is still in progress. Had submission blocked until the answer was ready, the task IDs and the polling loop would have been decorative. Our test suite asserts this directly: it submits with an artificial delay active and checks the acknowledgment returns in under a second.

The acknowledgment always reports `submitted`. We found that the worker thread could move the task to `working` before the `201` response was serialized, meaning a submission could return a status that contradicted our own contract. The acknowledgment is now an acceptance snapshot that always reports `submitted`, independent of what the stored task has already become.

The question field carries the user's words and nothing else. Our first version wrapped the issue in instruction text — *"Using the support knowledge base, identify the correct category..."* — and passed the whole string as the task question. Because the Specialist uses that string directly as its retrieval query, the instruction wording became noise in the query and diluted the real signal: three of the five test cases dropped below the relevance threshold and failed outright. We moved prompt framing into the RAG layer, which knows how it is querying, and the protocol now carries the user's text plus a separate `needs` field stating what the Requester wants back.

4. The RAG Pipeline

The pipeline runs in five stages over the seven provided documents.

1. **Load and chunk.** Each document is parsed into its markdown sections. Chunks are built only from the sections that describe *problems* — Description, Symptoms, and When to Use — rather than from the whole document.
2. **Index twice.** A sparse BM25 index and a dense index using `all-MiniLM-L6-v2` embeddings in FAISS, both over the same chunks. The built index is cached to disk, keyed by a SHA-256 fingerprint of the corpus, so it is rebuilt only when the documents change.
3. **Fuse.** Both retrievers produce a ranking, and the two rankings are combined with Reciprocal Rank Fusion at `k = 60`, keeping the top eight candidates.
4. **Grade.** An LLM is asked, in one batched call, whether each candidate passage is actually relevant to the question. Chunks that fail are discarded. If nothing survives, the task fails with `NO_RELEVANT_CONTEXT`.
5. **Generate.** The winning document's category is read from its own `Ticket Category` section, and the LLM writes one to three sentences of resolution notes constrained to the document's `Resolution` text, at temperature 0.

Advanced technique: fusion retrieval with relevance grading

Our chosen technique is **fusion retrieval** — hybrid sparse and dense retrieval combined with Reciprocal Rank Fusion — paired with **LLM relevance grading** before generation. Both sit outside the foundational techniques.

The choice was driven by a specific property of this corpus. The seven documents share a great deal of boilerplate: the instruction "Restart the computer if appropriate" appears in four of them, and every file follows the same Description / Symptoms / Troubleshooting / Resolution skeleton. Dense embeddings capture that shared structure and blur documents that are semantically adjacent but practically distinct, and confusing `network.md` with `hardware.md` puts the wrong value in the dropdown. BM25 preserves exactly the discriminating signal that embeddings smooth away — the literal tokens "Wi-Fi" and "keyboard". Fusing both rankings keeps the keyword precision while retaining the semantic reach that connects "cannot log in" to "account locked".

Relevance grading was added for a second reason. Without it, a question the knowledge base does not cover still returns the closest chunk, and the LLM will write a plausible answer from it. Grading gives the pipeline a principled way to say *nothing here answers this*, which is what makes our `NO_RELEVANT_CONTEXT` failure path honest rather than cosmetic.

Supporting decisions

- **Chunk on problem sections only.** Retrieval matches a user's symptom description against symptom text, not against resolution text. Including Resolution sections in the index made every document look similar, since the resolutions are written in the same register.
- **Read the category, generate the resolution.** The category is taken verbatim from the document rather than generated, because it must match a fixed set of options exactly and there is no reason to let a model paraphrase it. Only the prose resolution is generated.
- **One batched grading call.** Grading eight chunks in a single numbered prompt rather than eight separate calls, for latency and cost.
- **Sources are always returned**, so every answer can be traced to the documents it came from.

5. The Playwright Workflow

The mock application is a static page with no backend, so it is loaded over a `file://` URL built from the repository path. The submitter performs the following sequence:

1. Launch Chromium and open the page, waiting for `domcontentloaded`.
2. Fill `#issue` with the user's original request.
3. Select `#category` using the option value derived from the Specialist's category.
4. Fill `#resolution` with the Specialist's generated resolution notes.
5. Click `#submit-ticket`.
6. Wait for `#confirmation:not(.hidden)`. If it does not appear within the timeout, read `#error` from the page and raise `VERIFICATION_FAILED` carrying what the page actually said.
7. Read `#ticket-id`, `#ticket-category` and `#ticket-resolution`, and confirm the ticket ID is five digits.
8. Close the browser in a `finally` block.

Verification is a comparison, not a presence check

A confirmation box appearing only proves the form accepted something. The coordinator additionally compares the category and resolution echoed back by the page against what the Specialist returned, and fails the run if either differs. This means a ticket filed under the wrong category cannot be reported as a success.

The category mapping, and why it can refuse

The Specialist returns categories in the knowledge base's own wording. The Requester maps these onto the four values the dropdown accepts:

Knowledge base category	Form option
Account Access	<code>account_access</code>
Hardware	<code>hardware</code>
Software	<code>software</code>
Network	<code>network</code>
Email	no option exists
Security	no option exists

The knowledge base documents Email and Security, and the provided test cases include an email synchronisation issue whose expected category is "Email", but the form's dropdown offers no such option. Rather than silently choosing the nearest available category, the Requester raises `UNMAPPABLE_CATEGORY` and declines to submit. The mapping is checked before a browser is launched, so this failure is fast and cheap. We treat this as a correctness property: a ticket filed under a knowingly wrong category is worse than a ticket not filed.

6. Error Handling and Failure Modes

Scenario	Cause	System behaviour
NO_RELEVANT_CONTEXT	No retrieved passage graded relevant to the question.	Task ends <code>failed</code> with the code. The Requester reports it and never opens a browser.
TIMEOUT	Task stayed non-terminal past the client deadline.	The Requester stops polling and reports how long it waited and how many times it polled.
UNMAPPABLE_CATEGORY	Specialist returned a category the form cannot express.	Submission is refused before the browser launches; the message names the category and the options available.
LLM_ERROR	Model call failed, or returned output that would not parse.	Any worker exception is converted into a <code>failed</code> task rather than crashing the server.
VERIFICATION_FAILED	Confirmation did not appear, or did not match the Specialist's answer.	Reported as a failure carrying the page's own error text.
SPECIALIST_UNREACHABLE	The Specialist process is not running.	Reported immediately instead of hanging.
UNKNOWN_TASK	A polled task ID does not exist.	<code>404</code> with the code; the client stops polling.

Two structural choices make these tractable. The coordinator never raises: every failure returns as a structured outcome carrying an error code, which is why each scenario above is reproducible on demand. And any unexpected worker exception is caught and converted into a `failed` task, so a bug in retrieval degrades one ticket rather than taking down the agent.

7. Results

All five provided test cases complete the full A2A round trip. The table reports tickets successfully submitted and categories matching the expected value.

Configuration	Submitted	Correct category
Baseline keyword retriever	4 / 5	4 / 5
Fusion retrieval + grading	—	—

To complete before submission: run `RAG_BACKEND=real python -m requester.cli --all` and fill in the second row. The baseline row is measured; the fusion row must be measured the same way rather than asserted.

The baseline's one categorisation error is instructive. Given "My laptop won't connect to Wi-Fi", keyword overlap ranked `hardware.md` above `network.md`, because "laptop" matches hardware symptom text more strongly than "Wi-Fi" matches network symptom text. This is precisely the weakness fusion retrieval was chosen to address.

The fifth case, an email synchronisation issue, is correctly classified as Email by both configurations and then correctly refused at the form boundary, because the provided application has no Email option. We count this as intended behaviour rather than a failure of the pipeline.

8. Reflection

What worked

Writing the protocol and the result payload down before writing code was the single most useful decision we made. It meant four people could work on four components concurrently, each against a stub of the others, and the pieces fit when they met. The two plug points — injecting the RAG pipeline as a worker function and hiding the browser behind a `Submitter` interface — were what made that concurrency real rather than theoretical: the entire retrieval layer was replaced without a single change to the agent that calls it.

Making the coordinator return structured outcomes instead of raising exceptions also paid off more than expected. Every failure mode became something we could demonstrate on demand rather than something we had to describe.

What did not work at first

Our initial design put the instruction prompt into the A2A message, which polluted the retrieval query and silently broke three of five test cases. The lesson was that in a RAG system the retrieval query and the generation prompt are different objects and belong in different layers.

We also had a protocol state that was defined but unreachable. Our artificial delay was placed before the transition to `working` rather than after it, so tasks moved from `submitted` straight to `completed` and the `working` state never appeared in any poll. It looked correct in the code and was wrong in practice, which is what convinced us to write tests that assert each state is actually observable rather than merely defined.

On RAG effectiveness

Retrieval quality on this corpus is dominated by how similar the documents are to each other, not by how much text there is. With seven near-identically structured documents, the difficulty is discrimination rather than recall, which is why a hybrid of keyword and semantic retrieval outperformed either alone and why chunking on problem sections mattered more than chunk size. We would expect a much larger corpus to shift that balance.

Limitations

- Tasks are held in memory, so a restart loses them. A real deployment would need persistence.
- There is no retry on a failed LLM call, so a transient model error fails an otherwise valid ticket.
- Cancellation is not implemented, so a timed-out task keeps running and completes into a result nobody reads.
- The confidence score is a retrieval statistic, not a calibrated probability, and should not be interpreted as one.
- The Requester is hardcoded to one Specialist; there is no discovery mechanism.

What we would improve

- Add retries with backoff for transient failures, distinguishing them from genuine task failures.
- Persist tasks, and add cancellation so abandoned work stops.
- Publish an agent card so the Requester could discover Specialists and route by skill, which is what a second expert agent would require.
- Build an evaluation harness that scores retrieval on more than five cases, since five is too few to tune a threshold against with confidence.
- Resolve the Email and Security categories properly — either by extending the form or by defining an escalation path — rather than treating them only as a refusal.

9. Team Contributions

Member	Contribution
Parth Auti	Requester Agent and Specialist Agent (steps 1–2); the contracts document; project structure and contribution workflow.
Hasti Bhalodi	A2A protocol (step 3): task lifecycle, state machine, error taxonomy, and the protocol test suite.
Medha Jani	RAG pipeline and advanced technique (step 4): chunking, dual indexing, Reciprocal Rank Fusion, relevance grading, and generation.
Kento Hopkins	Playwright automation and verification (steps 5–6), and browser-side failure handling.

Appendix — Running the System

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
playwright install chromium
cp .env.example .env          # add GROQ_API_KEY for the real pipeline

# terminal 1
python -m specialist.server --delay 1

# terminal 2
python -m requester.cli --all
python -m unittest test_a2a_protocol -v
```

The timeout scenario is reproduced by starting the Specialist with `--delay 999` and running the Requester with `--timeout 3`. The `NO_RELEVANT_CONTEXT` scenario is reproduced by submitting a question the knowledge base does not cover, such as "What is the capital of France?".